

Unified Monitoring and SIEM Lab: Zabbix, Wazuh and Centralised Syslog

By

Mohamed Irneen Ahmed

TP085087

(14/10/2026)

TABLE OF CONTENTS

1.0 Introduction 3

2.0 System Design 3

2.1 Objectives and Scope 3

2.2 Architecture 4

2.3 Tools and Versions 5

3.0 Implementation 6

3.1 Lab Network and Virtual Machines 6

3.2 Wazuh 6

3.3 Zabbix 8

3.4 Telegram Alert Hub 9

3.5 Syslog Relay 10

3.6 OPNsense Firewall 11

3.7 SNMP Monitoring 13

4.0 Testing and Results 14

4.1 Test Summary 14

4.2 End-to-End Walkthrough 15

5.0 Issues and Resolutions 15

6.0 Discussion and Limitations 16

7.0 Conclusion 16

8.0 References 17

1.0 INTRODUCTION

Organisations normally run two kinds of monitoring. Availability monitoring answers whether a device is up and healthy, and security monitoring answers whether something suspicious is happening to it. When the two live in separate tools with separate alert channels, an operator has to correlate events by hand. This project builds a small unified lab that combines Zabbix for availability (Zabbix, n.d.) and Wazuh for security detection (Wazuh, n.d.). Both are fed by the same logs, and both report to a single Telegram chat.

The whole lab runs in VMware Workstation Pro on one laptop, using five virtual machines. Servers and a firewall send syslog to a central relay, which forwards the stream to both platforms, and Zabbix also polls devices over SNMP. This report describes the design, how each component was configured, the tests used to verify it, the problems met along the way, and the limitations of the lab. All attack simulations were run against the author's own virtual machines.

2.0 SYSTEM DESIGN

2.1 OBJECTIVES AND SCOPE

The aim was to run Zabbix alongside Wazuh, forward syslog from servers and a firewall to both, add SNMP monitoring of network devices, and bring all alerts into one place. Zabbix was chosen over LibreNMS because of its more flexible trigger and alerting engine. Table 1 shows how each objective was met.

Objective	How it was met
Zabbix running next to Wazuh	Both installed and monitoring the same host (linux-host)
Syslog from servers and a firewall to both platforms	An rsyslog relay forwards each message to Wazuh and stores it for Zabbix
SNMP monitoring of network devices	snmpd on a Linux host and the Net-SNMP plugin on OPNsense, polled by Zabbix
One alert hub for uptime and threats	A Telegram bot receives Wazuh and Zabbix alerts in the same chat

Table 1: Objectives and how they were met

2.2 ARCHITECTURE

All machines share a private Host-only network (192.168.115.0/24) and each also has a NAT adapter for internet access. Figure 1 shows the design. Log sources (linux-host and the OPNsense firewall) send syslog over UDP port 514 to the relay. The relay forwards a copy to the Wazuh manager and writes every message to a file that a Zabbix agent watches. Wazuh agents report directly to the manager, and Zabbix polls SNMP devices and agents.

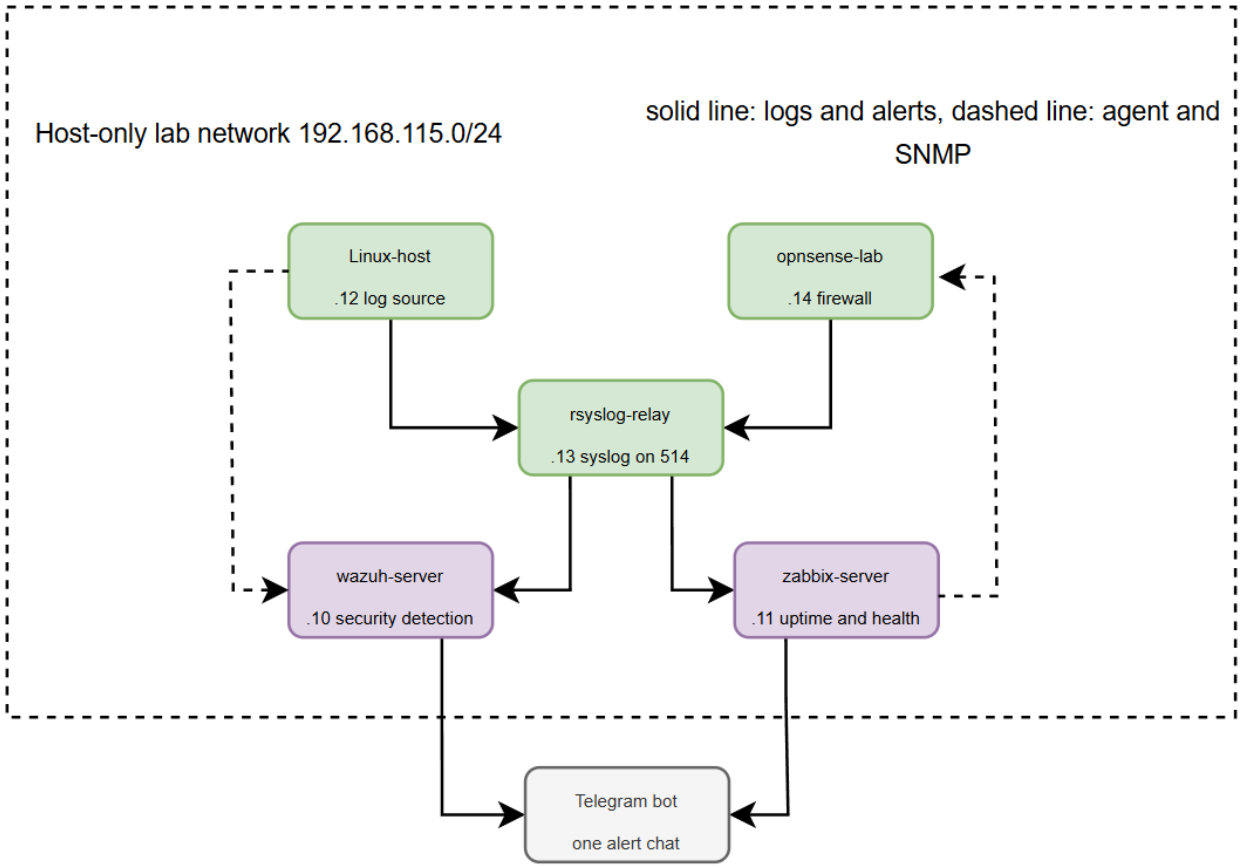
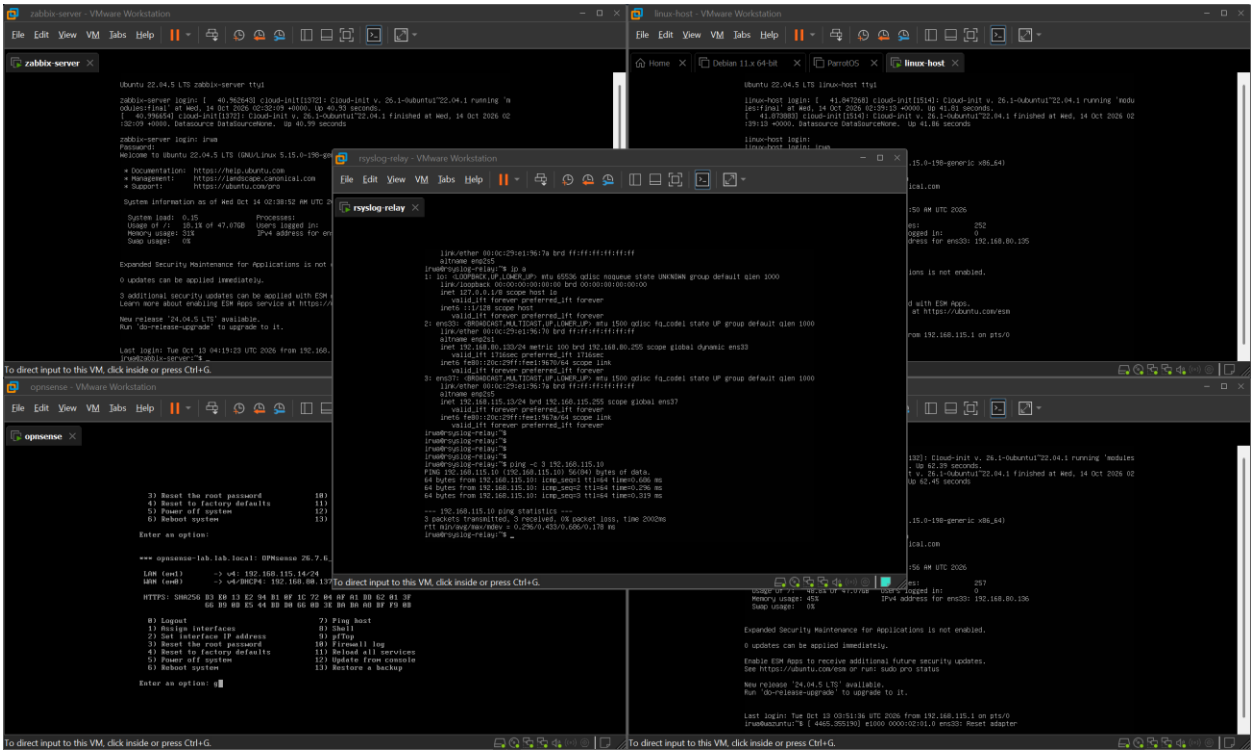


Figure 1: Architecture of the unified monitoring and SIEM lab

VM	Host-only IP	Role	RAM
wazuh-server	192.168.115.10	Wazuh manager, indexer and dashboard (all-in-one)	6 GB
zabbix-server	192.168.115.11	Zabbix server, web frontend and MySQL database	3 GB
linux-host	192.168.115.12	Monitored server: Wazuh agent, Zabbix agent, snmpd	1.5 GB
rsyslog-relay	192.168.115.13	Central syslog relay and Zabbix agent	1 GB
opnsense-lab	192.168.115.14	Firewall: syslog source and SNMP device	2 GB

Table 2: Virtual machine inventory



Running 5 concurrent vms (RIP RAM)

2.3 Tools and Versions

The lab uses Ubuntu Server 22.04.5 LTS for the Linux machines, Wazuh 4.14.8, Zabbix 7.0 LTS with MySQL and Apache, OPNsense 26.7.6_3, rsyslog, Net-SNMP, and the Telegram Bot API (Telegram, n.d.). The host is a laptop with 32 GB of RAM running VMware Workstation Pro.

3.0 IMPLEMENTATION

3.1 LAB NETWORK AND VIRTUAL MACHINES

One Ubuntu Server VM was installed and snapshotted, then cloned for the other machines. Each clone received a new hostname, machine ID, SSH host keys and static IP. Cloud-init was told not to manage the network, because it regenerated the netplan file at boot and removed the static address (Canonical, n.d.).

```
# /etc/netplan/50-cloud-init.yaml (example: linux-host)
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: true
    ens37:
      dhcp4: false
      addresses:
        - 192.168.115.12/24
```

```
irwa@linux-host:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:98:9b:06 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.80.135/24 metric 100 brd 192.168.80.255 scope global dynamic ens33
       valid_lft 1191sec preferred_lft 1191sec
    inet6 fe80::20c:29ff:fe98:9b06/64 scope link
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:98:9b:10 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 192.168.115.12/24 brd 192.168.115.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe98:9b10/64 scope link
       valid_lft forever preferred_lft forever
```

Figure 2: Virtual machine library and static IP configuration

3.2 WAZUH

Wazuh was installed on wazuh-server with the all-in-one installation assistant, which sets up the manager, indexer and dashboard on one machine. The first attempt failed at the dashboard

stage because the root volume was only half of the disk, so it was extended with LVM and the install repeated. An agent was then deployed to linux-host using the dashboard's deployment wizard.

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh \
  && sudo bash ./wazuh-install.sh -a


sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
```

To test detection, repeated failed SSH logins were made from wazuh-server to linux-host with a non-existent user. Wazuh raised level 10 alerts (rules 2502 and 5712) mapped to MITRE ATT&CK technique T1110, Brute Force (MITRE, n.d.).

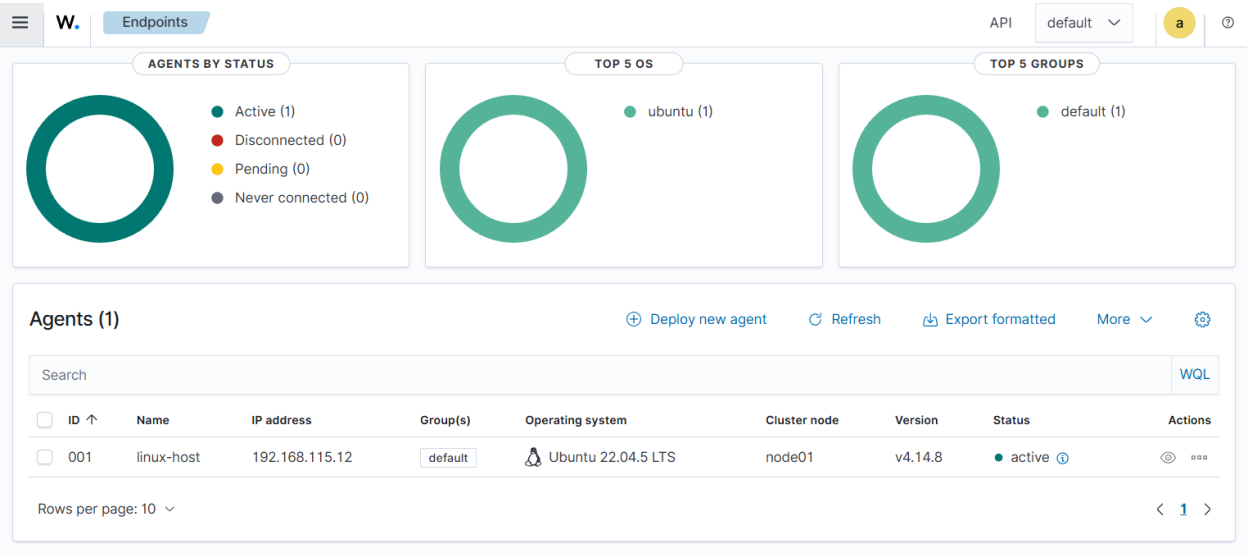


Figure 3: Wazuh Endpoints Summary showing linux-host as Active

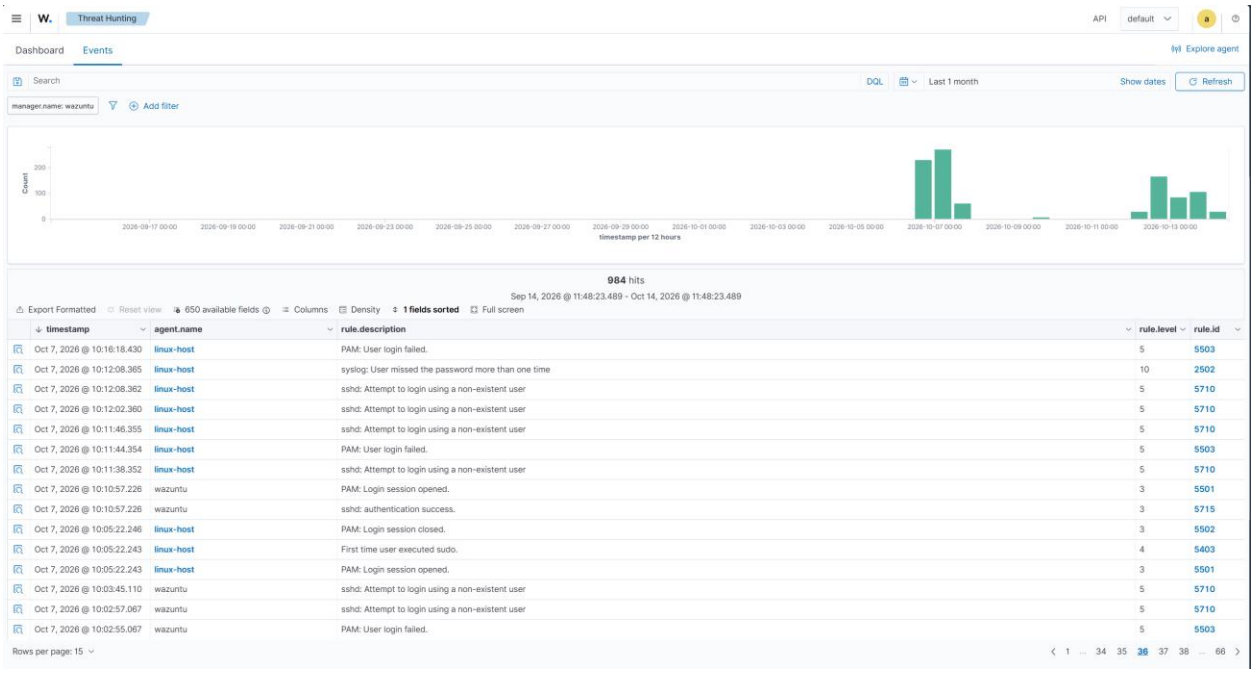


Figure 4: Wazuh event details for the SSH brute-force alert

3.3 ZABBIX

Zabbix 7.0 was installed from the official repository with the server, web frontend, agent, MySQL database and Apache. The database schema was imported and the web setup wizard completed. Zabbix agents were installed on linux-host and rsyslog-relay and linked to the template Linux by Zabbix agent, which includes a trigger for an unreachable agent (Zabbix, n.d.).

```
# /etc/zabbix/zabbix_agentd.conf (linux-host)
Server=192.168.115.11
ServerActive=192.168.115.11
Hostname=linux-host


# availability test
sudo systemctl stop zabbix-agent
```

Stopping the agent produced the problem “Zabbix agent is not available (for 3m)” about three minutes later, which cleared after the agent was restarted.

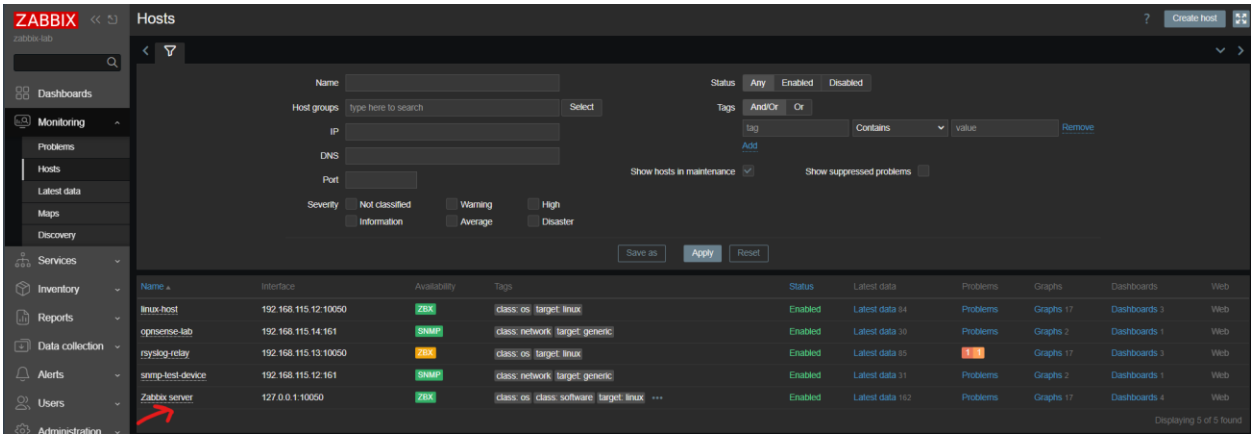


Figure 5: Zabbix Latest data for linux-host with the agent reporting

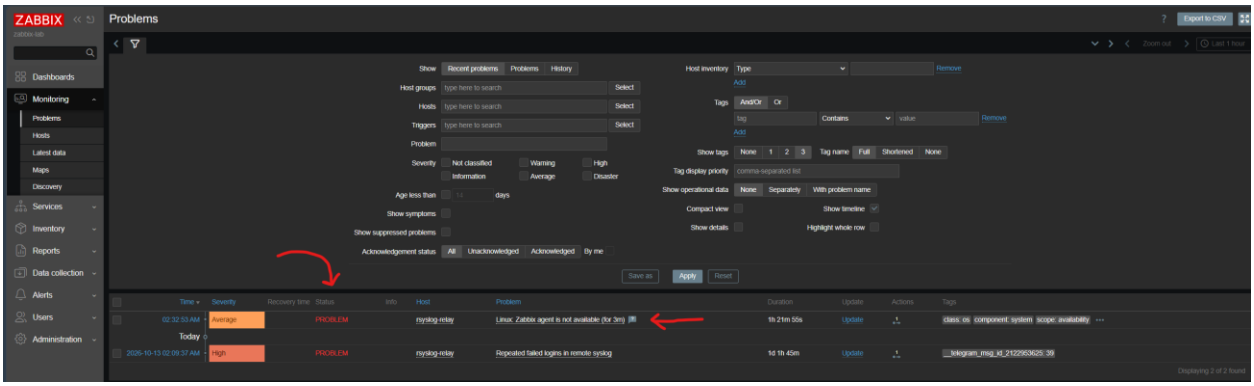


Figure 6: Zabbix problem for the unavailable agent, and its resolution

3.4 TELEGRAM ALERT HUB

A Telegram bot was created with BotFather. Zabbix uses its built-in Telegram media type, with the bot token in the api_token parameter, the chat ID added to the Admin user's media, and the default problem action enabled. Wazuh has no built-in Telegram option, so a custom integration script (custom-telegram) was added to the manager. It runs for every alert at level 10 or above and posts the rule, level, agent and raw log line to the bot.

```
# /var/ossec/etc/ossec.conf (token and chat ID removed)
<integration>
  <name>custom-telegram</name>
  <hook_url>https://api.telegram.org/bot<TOKEN>/sendMessage?chat_id=<ID></hook_url>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>
```

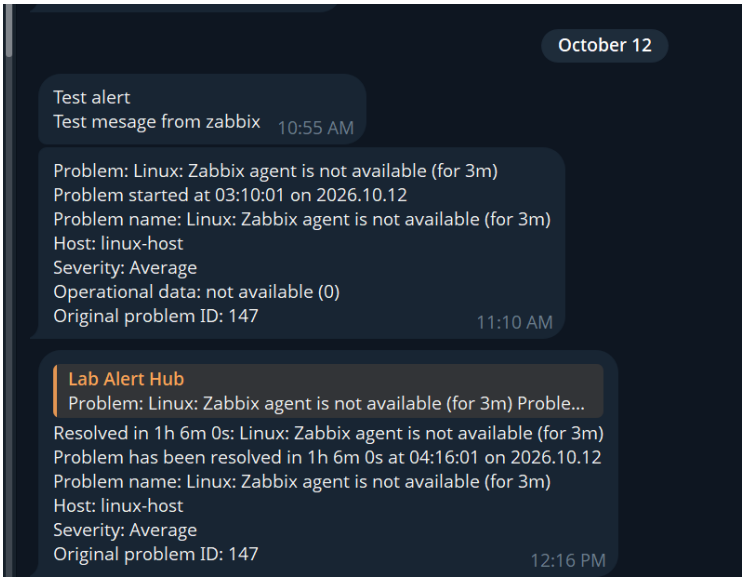


Figure 7: Telegram chat showing a Wazuh alert and a Zabbix alert together

3.5 SYSLOG RELAY

The relay uses rsyslog (rsyslog, n.d.) to listen on port 514, write each message to /var/log/remote/all.log, and forward a copy to Wazuh. Wazuh was configured to accept syslog from the relay's address only. Zabbix cannot receive syslog directly, so an active log item on the relay reads the stored file, and a trigger fires when five or more failed-password lines arrive within five minutes. A recovery expression closes the problem after five minutes without matches.

```
# /etc/rsyslog.d/10-relay.conf
module(load="imudp")
module(load="imtcp")
ruleset(name="remote") {
    action(type="omfile" file="/var/log/remote/all.log")
    action(type="omfwd" target="192.168.115.10" port="514" protocol="udp")
}
input(type="imudp" port="514" ruleset="remote")
input(type="imtcp" port="514" ruleset="remote")


# /var/ossec/etc/ossec.conf (Wazuh manager)
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.115.13</allowed-ips>
```

```
</remote>

# Zabbix item on rsyslog-relay (Zabbix agent active, type Log)
log[/var/log/remote/all.log,"Failed password"]


# Trigger and recovery expressions
count(/rsyslog-relay/log[/var/log/remote/all.log,"Failed password"],5m)>=5
nodata(/rsyslog-relay/log[/var/log/remote/all.log,"Failed password"],5m)=1
```

The chain was tested by sending eight fake failed-SSH messages from linux-host to the relay:

```
for i in $(seq 1 8); do logger -n 192.168.115.13 -P 514 -d -i -t sshd \
  'Failed password for invalid user hacker from 10.9.9.9 port 4444 ssh2'; done
```

```
irwa@rsyslog-relay:~$ sudo tail -5 /var/log/remote/all.log
[sudo] password for irwa:
Oct 14 12:06:00 opnsense-lab.lab.local /usr/sbin/cron[51567]: (root) CMD ((/usr/local/bin/flock -
n -E 0 -o /tmp/filter_update_tables.lock /usr/local/opnsense/scripts/filter/update_tables.py --qu
ick) > /dev/null)
Oct 14 12:06:00 opnsense-lab.lab.local /usr/sbin/cron[51618]: (root) CMD ((/usr/local/bin/flock -
n -E 0 -o /tmp/updaterrd.lock /usr/local/opnsense/scripts/health/updaterrd.php) > /dev/null)
Oct 14 12:06:16 opnsense-lab.lab.local filterlog[25829]: 72,,,1232f88e5fac29a32501e3f051020cac,em
0,match,pass,out,4,0xb8,,64,42716,0,none,17,udp,76,192.168.80.137,103.16.183.23,123,123,56
Oct 14 12:06:24 opnsense-lab.lab.local filterlog[25829]: 72,,,1232f88e5fac29a32501e3f051020cac,em
0,match,pass,out,4,0xb8,,64,39888,0,none,17,udp,76,192.168.80.137,47.254.196.78,123,123,56
Oct 14 12:06:29 opnsense-lab.lab.local filterlog[25829]: 72,,,1232f88e5fac29a32501e3f051020cac,em
0,match,pass,out,4,0xb8,,64,59124,0,none,17,udp,76,192.168.80.137,111.90.158.16,123,123,56
irwa@rsyslog-relay:~$
```

Figure 8: Relay log file showing the forwarded messages

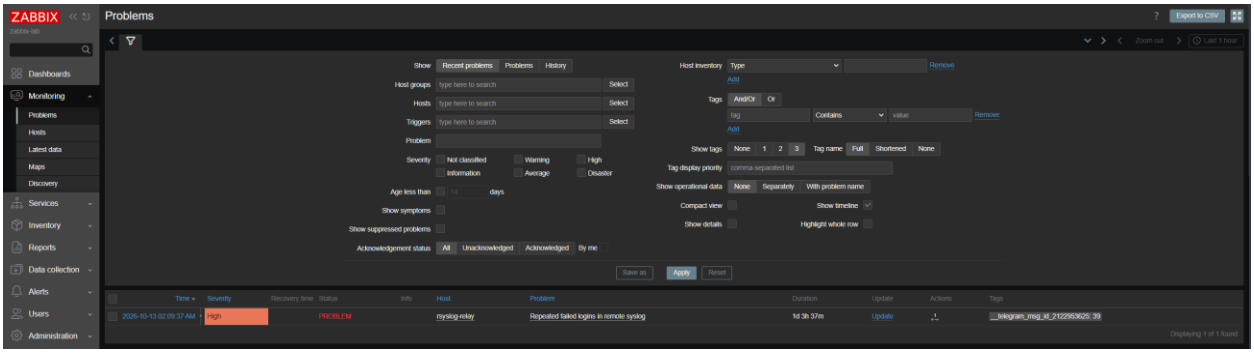


Figure 9: Zabbix problem raised by the remote syslog trigger

3.6 OPNsense Firewall

OPNsense was installed with its WAN on the NAT adapter and its LAN on the Host-only network at 192.168.115.14. A remote logging target (System > Settings > Logging > Remote) sends all logs to the relay over UDP port 514 (OPNsense, n.d.). Wazuh already includes rules for

firewall block events, and these produced the level 10 alert “Multiple pfSense firewall blocks events from same source” (rule 87702) when the firewall blocked repeated connections.

Failed OPNsense web logins, however, were decoded as Solaris BSM audit messages and matched a built-in level 0 rule (6100), so no alert was created. Two custom rules were added as children of rule 6100. Rule 100100 alerts on each failed login, and rule 100101 raises a level 10 alert after three failures within two minutes. The rules were verified with wazuh-logtest.

```
<group name="local,opnsense,authentication_failed,">
  <rule id="100100" level="5">
    <if_sid>6100</if_sid>
    <match>Web GUI authentication error</match>
    <description>OPNsense: Web GUI authentication failed.</description>
    <mitre><id>T1110</id></mitre>
  </rule>
  <rule id="100101" level="10" frequency="3" timeframe="120">
    <if_matched_sid>100100</if_matched_sid>
    <description>OPNsense: multiple Web GUI authentication failures.</description>
    <mitre><id>T1110</id></mitre>
  </rule>
</group>
```

During testing the firewall’s own brute-force protection added the laptop to its sshlockout table, which blocked access to the web interface. The block was cleared with pfctl, and it also served as a useful example of detection and automatic response working together.

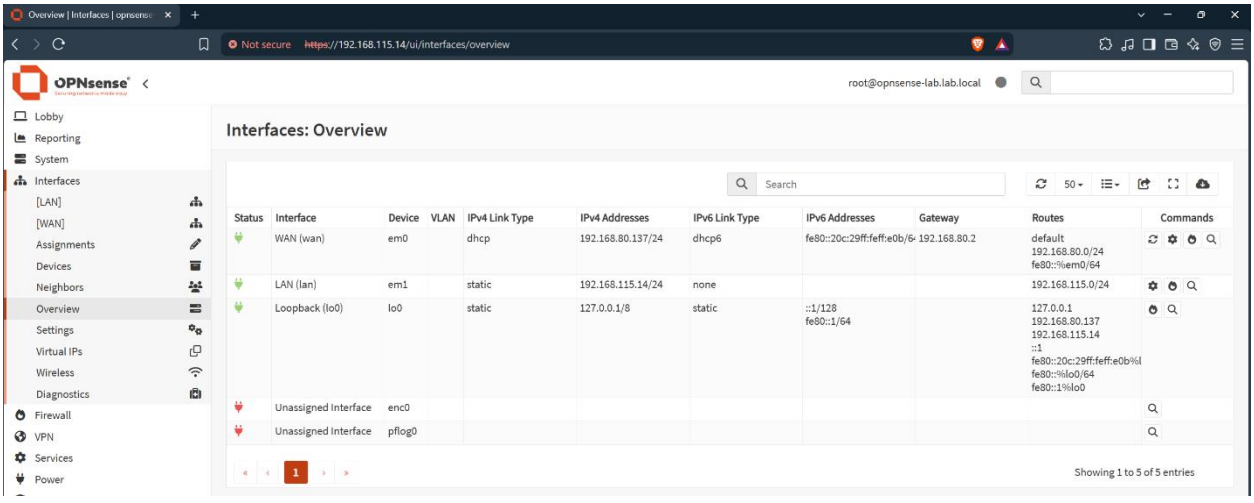


Figure 10: OPNsense interfaces overview (WAN on NAT, LAN 192.168.115.14)

```
irwa@wazuntu:~$  sudo /var/ossec/bin/wazuh-logtest
[sudo] password for irwa:
Starting wazuh-logtest v4.14.8
Type one log per line

Oct 13 12:02:54 opnsense-lab.lab.local audit[37278]: /index.php: Web GUI authentication error for
 'root' from 192.168.115.1

**Phase 1: Completed pre-decoding.
        full event: 'Oct 13 12:02:54 opnsense-lab.lab.local audit[37278]: /index.php: Web GUI aut
hentication error for 'root' from 192.168.115.1'
        timestamp: 'Oct 13 12:02:54'
        hostname: 'opnsense-lab.lab.local'
        program_name: 'audit'

**Phase 2: Completed decoding.
        name: 'solaris_bsm'

**Phase 3: Completed filtering (rules).
        id: '100100'
        level: '5'
        description: 'OPNsense: Web GUI authentication failed.'
        groups: '['local', 'opnsense', 'authentication_failed']'
        firedtimes: '1'
        mail: 'False'
        mitre.id: '['T1110']'
        mitre.tactic: '['Credential Access']'
        mitre.technique: '['Brute Force']'
**Alert to be generated.
```

Figure 11: wazuh-logtest output matching custom rule 100100

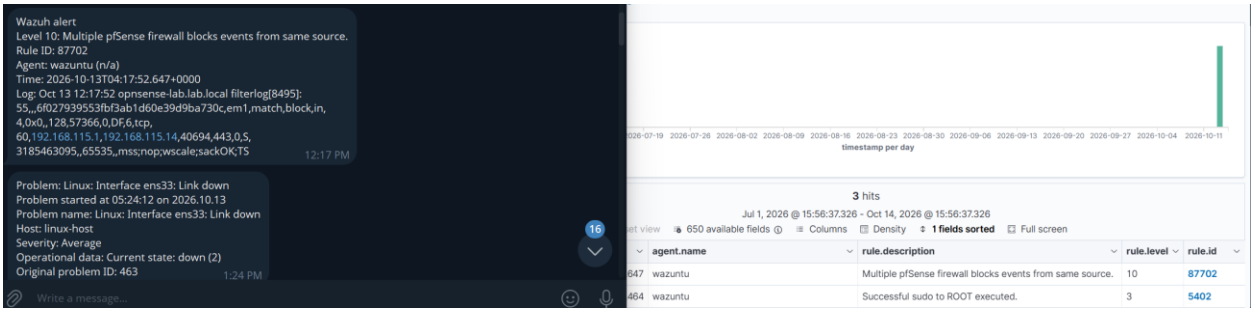


Figure 12: Wazuh and Telegram alert for firewall block events (rule 87702)

3.7 SNMP MONITORING

SNMP lets Zabbix monitor devices that cannot run an agent. Snmpd was installed on linux-host with a read-only community restricted to the Zabbix server, and the Net-SNMP plugin was installed on OPNsense after updating it to 26.7.6_3. Both devices were added to Zabbix with the template Network Generic Device by SNMP, which collects interface traffic, uptime and availability.

```
# /etc/snmp/snmpd.conf (linux-host)
```

```
agentaddress udp:161
rocommunity labmon 192.168.115.11
rocommunity labmon 127.0.0.1


# test from zabbix-server (returns the device name)
snmpget -v2c -c labmon 192.168.115.12 1.3.6.1.2.1.1.5.0
```

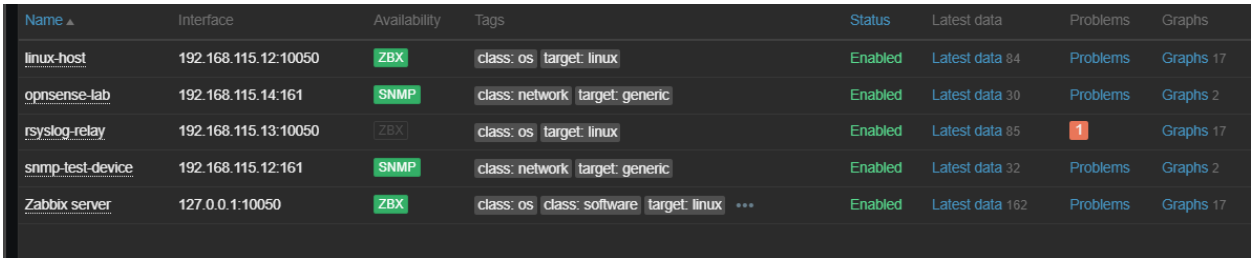


Figure 13: Zabbix SNMP hosts with interface traffic for the firewall

4.0 TESTING AND RESULTS

4.1 TEST SUMMARY

Table 3 lists the tests performed and their outcomes.

#	Test	Method	Result
1	Wazuh agent connected	Endpoints Summary	linux-host Active
2	SSH brute force detected	Failed logins to linux-host	Level 10 alert, T1110 (rules 2502, 5712)
3	Zabbix uptime alert	Stopped zabbix-agent	Problem after about 3 minutes, Telegram message received
4	Syslog chain	logger loop to the relay	Lines in all.log, Wazuh alert, Zabbix problem
5	Telegram alert hub	Wazuh and Zabbix alerts	Both received in the same chat
6	OPNsense logs reach the relay	Wrong web GUI password	audit lines stored in all.log
7	Custom rule 100100	wazuh-logtest with a real log line	Matched at level 5 with MITRE T1110
8	Firewall block detection	Repeated wrong web passwords	Rule 87702 level 10, Telegram message
9	SNMP polling	snmpget and Zabbix hosts	Device name returned; Zabbix hosts added

Table 3: Test results

4.2 END-TO-END WALKTHROUGH

Security event: A failed SSH login on linux-host is written to its authentication log. The Wazuh agent forwards it, the manager matches it to a rule, and the resulting alert is mapped to MITRE ATT&CK and sent to Telegram when its level is 10 or above.

Same event through the relay: A syslog message sent to the relay is stored and forwarded. Wazuh decodes the forwarded copy, while the Zabbix agent counts matching lines in the stored file, so one simulated attack is seen as a threat in Wazuh and as a threshold breach in Zabbix.

Availability event: Stopping the Zabbix agent creates a Zabbix problem only. This is an availability issue and not a security event, which shows why the two platforms complement each other.

5.0 ISSUES AND RESOLUTIONS

Problem	Cause	Resolution
Wazuh dashboard installation failed with a disk full error	LVM gave the root volume only half of the disk	Extended it with lvextend and repeated the install
Static IP disappeared after reboot	cloud-init regenerated the netplan file	Disabled cloud-init network configuration
Zabbix log item showed Not supported	Item was created on the Zabbix server host instead of rsyslog-relay	Copied the item to the correct host
Zabbix problem never cleared	count() trigger stuck when no new log lines arrived	Added a nodata() recovery expression
Wazuh ignored OPNsense login failures	Built-in level 0 rule 6100 matched first	Wrote custom rule 100100 as a child of rule 6100
Locked out of the OPNsense web UI	Brute-force protection added the laptop to sshlockout	Flushed the table with pfctl
Cloned VMs clashed	Clones shared IP address and SSH host keys	New hostname, machine ID, SSH keys and IP per clone

Table 4: Problems encountered

6.0 DISCUSSION AND LIMITATIONS

What the lab shows: Combining the two platforms gave faster understanding of an event than either would alone. Wazuh explained what was suspicious and why, and Zabbix showed whether the system was still healthy. A central relay kept the design simple, since a new device only needs one syslog destination.

Limitations: This is a lab and not a production design. SNMP version 2c community strings and UDP syslog are unencrypted and unreliable, the bot token is stored in a configuration file, and alerts are not deduplicated, so one attack can generate several messages. Only one Linux server and one firewall were monitored, and the network devices were not physical switches.

Future work: Improvements include SNMPv3, syslog over TCP with TLS, Wazuh active response for automatic blocking, physical or emulated switches in GNS3, and Zabbix dashboards that show Wazuh and Zabbix data side by side.

7.0 CONCLUSION

The project delivered a working unified monitoring and SIEM lab on a single laptop. Zabbix and Wazuh run side by side, syslog from a server and a firewall reaches both through one relay, SNMP polling covers network devices, and all alerts arrive in one Telegram chat. Testing with simulated brute-force attacks and an availability fault confirmed each part, and troubleshooting the gaps, such as the custom OPNsense rules, gave practical experience with real monitoring and detection problems.

8.0 REFERENCES

Canonical. (n.d.). *Netplan documentation*. https://netplan.readthedocs.io/

MITRE. (n.d.). *Brute force: Technique T1110 – Enterprise*. https://attack.mitre.org/techniques/T1110/

OPNsense. (n.d.). *OPNsense documentation*. https://docs.opnsense.org/

rsyslog. (n.d.). *rsyslog documentation*. https://www.rsyslog.com/doc/

Telegram. (n.d.). *Telegram Bot API*. https://core.telegram.org/bots/api

Wazuh. (n.d.). *Wazuh documentation*. https://documentation.wazuh.com/current/

Zabbix. (n.d.). *Zabbix documentation*. https://www.zabbix.com/documentation/current/